

INTERVIEW PREPARATION GUIDE

AI Lab Intern + PPO Hiring — Automatic Infotech, Pune

Prepared for: Pruthviraj Thorbole

B.Tech CSE, ADCET Ashta | CPI 8.81

Communication Round → Technical Interview 1 → Technical Interview 2 → Project Assignment → Final Offer

Table of Contents

Table of Contents	2
1. How To Use This Guide — You Have Very Little Time	4
2. Interview Snapshot & Company Context	4
The role, in numbers	4
About Automatic Infotech	4
Logistics Checklist.....	5
3. Round 1 — Communication Round.....	6
What they're actually evaluating	6
Your “Tell Me About Yourself” — a working script.....	6
Core questions to walk in with answers for	6
The STAR method — use this for every “tell me about a time...” question.....	7
If there's a Group Discussion component	7
Delivery tips — often more decisive than content	7
4. Rounds 2 & 3 — Technical Interviews: Core Fundamentals	8
Python	8
SQL	8
Data Structures & Algorithms (basic level, matching your resume)	8
Object-Oriented Programming	9
DBMS Fundamentals.....	9
Git & GitHub.....	9
Cloud Basics (AWS).....	9
APIs, REST & JSON	9
5. Technical Interviews — AI/LLM Focus (the heart of this role).....	11
What is a Large Language Model (LLM)?	11
Prompt Engineering	11
Embeddings.....	11
RAG (Retrieval-Augmented Generation).....	11
LLM Providers & AI Coding Tools (named directly in the JD).....	12
A few extra scenario questions worth rehearsing out loud.....	12
6. Project Deep-Dive — Anticipated Questions on Your Own Projects	13
FaithGuard — AI-Assisted Lost & Found System	13
Cloud Image Recognition System.....	13
Sales Data Analysis Dashboard	13
7. Round 4 — Project Assignment	15
How to approach any assignment you're given.....	15
8. How To Stand Out — Communication & Presence.....	16
How to “justify” your answers so they're convincing	16
General standout habits.....	16

9. Smart Questions To Ask Them	17
10. Final Night-Before / Morning-Of Cheat Sheet	18
Your 30-second pitch	18
RAG in one breath.....	18
Prompt engineering in one breath.....	18
Your three projects, one line each.....	18
Two rules to remember walking in	18

(If the section list above shows blank, right-click it and choose "Update Field" — Word will fill it in instantly.)

1. How To Use This Guide — You Have Very Little Time

You said you're short on time, so here is the priority order if you can't get through everything below:

- Project Deep-Dive (Section 6) — the most personalized section and the most likely to actually come up, since it's built directly from your own resume.
 - AI/LLM Focus (Section 5) — this is the real differentiator for an “AI Lab” role. Basic fluency here puts you ahead of most fresher candidates.
 - Your “Tell Me About Yourself” script (Section 3) — read it out loud 3 times before you sleep.
 - Core Technical Fundamentals (Section 4) — skim it, this is mostly a refresher of things you already know from coursework.
 - Sections 7–9 (Stand Out, Smart Questions, Final Cheat Sheet) — read once, right before you leave for the interview.
-

2. Interview Snapshot & Company Context

The role, in numbers

- Role: AI Lab Intern (with PPO — pre-placement offer on conversion)
- Location: Pune
- Stipend during internship: ₹15,000/month
- CTC on conversion: ₹3.5 LPA
- Service agreement: 3 years
- Process: Communication Round → Technical Interview 1 → Technical Interview 2 → Project Assignment → Final Offer

About Automatic Infotech

Automatic Infotech is an established Pune-based IT and SAP consulting company that has been operating since the mid-2000s, working with mid-size and large clients across automotive, pharma, and manufacturing sectors, as part of a larger group of companies. On public employee-review platforms, the company scores well overall — strong average ratings, and most employees say they would recommend working there — and reported interview difficulty is described as moderate rather than extreme.

What this means for you: their core business has traditionally been SAP and enterprise IT consulting. The “AI Lab” looks like a newer, growing division inside a traditional IT services company. That means they are likely looking for someone who can bridge solid software engineering fundamentals with emerging AI/LLM skills — which is exactly your profile: full-stack development, data analytics, and growing AI/ML exposure through your Google AI-ML internship. Frame your answers around that bridge wherever you can.

What past candidates have reported: interviews have typically included practical questions on core programming fundamentals (Java/JavaScript basics) plus detailed discussion of the projects listed on the candidate's own resume, sometimes alongside a short written or MCQ-style technical screening. This lines up closely with how this guide is structured.

One practical note, worth taking seriously: multiple past candidates specifically mentioned it is worth fully understanding the 3-year service agreement and its exit/termination terms before signing anything. This is not unusual for Indian IT services hiring and is not a red flag by itself — it just means you should ask about it directly and clearly (a specific question for this is included in Section 9) so you're deciding with complete information, rather than under time pressure on offer day.

Logistics Checklist

- ☐ Reach the venue on time — Automatic Infotech, Kapil Upavan Plaza, Bibwewadi, Pune – 411037 (3rd floor, use the left-side lift, next to Lenskart)
- ☐ Carry an updated, printed resume (2–3 copies)
- ☐ Carry your college ID card
- ☐ Carry academic documents/marksheets — originals plus photocopies
- ☐ Wear formal attire
- ☐ Carry a pen and a small notepad
- ☐ Keep your phone on silent once inside
- ☐ Arrive 15–20 minutes early — use the buffer to breathe, not to cram
- ☐ Keep Section 10 (Final Cheat Sheet) open on your phone for one last look in the waiting area

3. Round 1 — Communication Round

What they're actually evaluating

Not just your English fluency. They're checking whether you can explain technical work clearly to someone who may not dig into every detail, whether you sound composed and coherent under mild pressure, and — importantly — whether you genuinely understand everything written on your own resume.

Your “Tell Me About Yourself” — a working script

I'm a third-year Computer Science and Engineering student at ADCET, Ashta, currently holding a CPI of 8.81. Over the last year I've built strong foundations in Python and full-stack web development, and I've been actively growing into AI/ML — I'm currently in Cohort 17 of the Google AI-ML Virtual Internship. I like to learn by building: I've shipped three projects, including FaithGuard, a community lost-and-found platform using React, Firebase, and Vertex AI, and a Cloud Image Recognition desktop app using the Google Cloud Vision API. I also completed a full-stack development internship at PrepGrad this summer. Outside academics, I founded The IT Crew Club, a 30-plus member placement-prep community at my college. I'm excited about this AI Lab Intern role specifically because it lets me combine my full-stack and data background with hands-on LLM and RAG work, which is exactly the direction I want to keep growing in.

Don't memorize this word-for-word — memorized answers sound memorized, and that's an instant flag for an interviewer. Read it 2–3 times, then say it in your own words each time you practice. The structure (present → experience → why this role) matters more than the exact sentences.

Core questions to walk in with answers for

Likely question: *Why should we hire you for this role?*

- Anchor on evidence, not adjectives. You've shipped three end-to-end projects with real feature counts — 5+ modules in FaithGuard, 4+ image-analysis features in your Cloud Vision app, 6+ visualizations in your sales dashboard. That's execution, not just theory.
- You're already building the exact bridge this role needs: full-stack + data analytics + a live AI/ML internship.
- You've shown initiative outside the classroom — founding The IT Crew Club and organizing placement training for 30+ students is leadership evidence most fresher candidates don't have.

Likely question: *What are your strengths?*

- Pick two, not five. Suggested: (1) fast learner across a new stack — you picked up Firebase, Vertex AI, Google Cloud Vision API, and Power BI across three different projects; (2) ownership — founding and running a 30-member community takes follow-through, not just an idea.

Likely question: *What's your weakness?*

- Be honest and specific, then show the fix already in motion. One genuine option based on your resume: your DSA and algorithms depth is still basic because you've spent more time shipping full projects than doing competitive programming, so you've started dedicating structured daily time to core data structures and algorithm patterns. Adjust this to whatever is actually true for you — don't recite it if it isn't.

Likely question: *Why do you want to join Automatic Infotech / this AI Lab Intern role?*

- Connect three things: (1) it's hands-on AI/LLM work, exactly the direction you're already growing in through your Google AI-ML internship; (2) it lets you apply your existing full-stack and data skills immediately instead of starting from zero; (3) the PPO structure means you can prove yourself and grow into a real role, not just finish an internship and leave.

Likely question: *Where do you see yourself in three years?*

- This one matters more than usual here because of the 3-year service agreement — they're partly listening for whether you're thinking long-term or just saying what sounds good. Answer honestly: growing from an intern into someone who can independently own AI/ML features, ideally within this company given the bond, while building deeper expertise in LLM applications, RAG systems, and production ML.

Likely question: *Tell me about a time you showed leadership or handled a challenge.*

- Use STAR (below). You have three strong real stories available: founding The IT Crew Club from scratch, serving as Training & Placement Coordinator for CESA, or a specific technical obstacle from one of your three projects — for example, getting QR-based anonymous authentication working in FaithGuard without a traditional login system. Pick whichever you can describe in the most concrete detail, with a specific number or outcome attached.

The STAR method — use this for every “tell me about a time...” question

- Situation — one or two sentences of context.
- Task — what you specifically were responsible for.
- Action — what YOU did. Say “I,” not just “we” — be specific about your own role.
- Result — the outcome, ideally with a number: members recruited, features shipped, bugs fixed, time saved.

Worked mini-example using the IT Crew Club: Situation — you noticed classmates lacked structured placement preparation. Task — self-initiated, no formal mandate. Action — founded The IT Crew Club and organized aptitude sessions, group discussions, and mock interviews. Result — grew the club to 30+ active members.

If there's a Group Discussion component

Some “communication rounds” include a GD. If yours does: speak early but with something substantive, reference specific facts or examples rather than generalities, build on others' points by name (“Adding to what X said...”), and help bring the discussion to a close with a short summary if no one else does. Don't dominate or interrupt — a GD is assessing collaboration, not just confidence.

Delivery tips — often more decisive than content

- Structure every answer: a one-sentence direct answer first, then 2–3 supporting sentences. Don't open with a long wind-up.
- A 2–3 second pause before answering a hard question is completely fine — it reads as thoughtful, not weak.
- Use concrete numbers wherever your resume has them — 5+ modules, 10+ SQL queries, 6+ visualizations, 30+ members. Numbers make claims verifiable and memorable.
- If it's a phone or video round: quiet room, charged device, decent lighting facing you, and look at the camera (not the screen) when you speak so it reads as eye contact.

4. Rounds 2 & 3 — Technical Interviews: Core Fundamentals

Companies vary in how they split two technical rounds — sometimes one is fundamentals and the other is projects/AI-focused, sometimes it's mixed throughout. Be ready across all of the below regardless of which round it lands in.

Python

- Mutable vs immutable: list/dict/set are mutable; tuple/string/int are immutable — know why it matters (e.g., you can't use a list as a dict key, but you can use a tuple).
- *args and **kwargs — variable-length positional and keyword arguments.
- List comprehensions — be ready to write one live, e.g. squares of even numbers under 10.
- Generators (yield) vs regular functions (return) — generators produce values lazily, one at a time, which saves memory on large sequences.
- Exception handling — try/except/else/finally, and when you'd raise a custom exception.
- Shallow copy vs deep copy — shallow copies nested objects by reference; deep copy duplicates everything recursively.
- Since you've used Pandas/NumPy: know why NumPy arrays outperform plain Python lists for numeric work (fixed type, vectorized operations, contiguous memory), and how you'd handle missing values in a DataFrame (dropna(), fillna()).

Likely question: *Walk me through the difference between a list and a tuple, and when you'd pick one over the other.*

- Tuples are immutable and hashable (usable as dict keys), so use them for fixed collections like coordinates; lists suit collections that change size or content.

SQL

- Joins: INNER (only matching rows), LEFT (all left rows + matches), RIGHT (all right rows + matches), FULL OUTER (all rows, both sides). Be ready to sketch a simple two-table example.
- GROUP BY with aggregate functions (COUNT, SUM, AVG, MAX, MIN); HAVING filters after grouping, WHERE filters before.
- Primary key vs foreign key; what an index does (speeds up lookups, at some cost to writes).
- Normalization — the general idea (reduce redundancy by splitting data into related tables) matters more here than reciting 1NF/2NF/3NF definitions verbatim.

Likely question: *You mention writing 10+ SQL queries to analyze regional sales performance — walk me through one of them.*

- Prepare one specific query from that project (e.g., top-N products by revenue per region using GROUP BY + ORDER BY + LIMIT) that you can explain end-to-end, including why you structured it that way.

Data Structures & Algorithms (basic level, matching your resume)

- Time complexity intuition: $O(1)$ constant, $O(\log n)$ like binary search, $O(n)$ single pass, $O(n^2)$ nested loops — reason about it out loud rather than proving it formally.
- Array vs Linked List — arrays give $O(1)$ indexed access but costly middle insertion/deletion; linked lists are the reverse.
- Hash map / Python dict — near $O(1)$ average lookup because of hashing.
- Stack (LIFO) vs Queue (FIFO) — with a plain example each: undo history is a stack; a print queue is a queue.
- Linear search vs binary search (binary needs sorted data, runs in $O(\log n)$).

If a question is outside your comfort zone: say what you do know, reason out loud, and ask a clarifying question rather than freezing or guessing silently. For fresher/intern roles, interviewers weight process and reasoning heavily.

Object-Oriented Programming

- Four pillars: Encapsulation (bundling data + methods, restricting direct access), Inheritance (a class reusing/extending another), Polymorphism (same interface, different behaviour depending on the object), Abstraction (hiding implementation detail behind a simple interface).
- Class vs Object — class is the blueprint, object is the instance.
- Worth mentioning since you list both Java and Python: Python doesn't support true method overloading (same name, different signatures) the way Java does — you'd use default arguments or *args instead.

DBMS Fundamentals

- ACID: Atomicity, Consistency, Isolation, Durability — know what each one protects against.
- SQL vs NoSQL, from your own experience: MySQL (relational, fixed schema, strong consistency) alongside MongoDB, Firebase, and Supabase (flexible schema, better suited to rapidly changing app data). Be ready to explain why you picked Firebase for FaithGuard over a traditional MySQL backend.

Git & GitHub

- Core commands: clone, add, commit, push, pull, branch, merge, status, log.
- What a merge conflict is and how you resolve one — Git flags the conflicting lines, you manually choose or combine the correct version, then commit.
- What a pull request is for — code review before merging into the main branch, relevant since your resume mentions collaborative coding practices at PrepGrad.

Cloud Basics (AWS)

Quick reference — know these at the level of “what it is and when you'd use it,” not deep configuration detail:

Service	What it is
EC2	Virtual servers / compute instances
S3	Object storage for files, images, backups
RDS	Managed relational database service
IAM	Identity & access management — users, roles, permissions
VPC	An isolated private network within AWS

Likely question: *Why would you use S3 instead of RDS?*

- S3 stores files/objects (images, documents, backups) cheaply and at scale; RDS is for structured, queryable relational data. Different jobs — often used together in the same system.

APIs, REST & JSON

- REST API: stateless, resource-based, built on standard HTTP methods.
- HTTP methods: GET (read), POST (create), PUT/PATCH (update), DELETE (remove).

- Common status codes: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found, 500 Internal Server Error.
- JSON: lightweight, human-readable, language-agnostic key-value format — why almost every modern API uses it.

Likely question: *You used the Google Maps API and Firebase Cloud Messaging in FaithGuard — how did your app authenticate with these APIs, and how did you handle the response data?*

- Have a real, specific answer ready: API keys, how you parsed the JSON response, and any rate-limit or error handling you built in.

5. Technical Interviews — AI/LLM Focus (the heart of this role)

This is the section that matters most for an “AI Lab Intern” role specifically — the JD names LLMs, prompt engineering, RAG, and embeddings directly. Solid conceptual fluency, even without deep production experience, will put you ahead of most fresher candidates who only prepared generic DSA.

What is a Large Language Model (LLM)?

A neural network built on the transformer architecture, trained on huge amounts of text to predict the next most likely word/token given what came before. That simple mechanism, at scale, produces the ability to write, summarize, translate, and reason across many tasks. Useful nuance to mention if asked: an LLM doesn't “look things up” the way a database does — it generates a statistically likely answer based on patterns learned during training, which is exactly why it can sound confident while being wrong (hallucination). That naturally leads into RAG below.

Prompt Engineering

- Zero-shot prompting: asking the model to do a task with no examples, just instructions.
- Few-shot prompting: giving 2–3 examples in the prompt so the model can infer the pattern you want.
- Chain-of-thought prompting: asking the model to reason step by step before giving a final answer — improves accuracy on multi-step problems.
- System prompt vs user prompt: the system prompt sets the model's role, behaviour, and constraints; the user prompt is the actual question or task.
- Good practices worth naming: be specific, specify the output format you want (e.g., “respond as a JSON object with these fields”), give context, and iterate — prompt engineering is empirical, you test and refine.

Embeddings

Numerical vector representations of text (or images) that capture meaning — two pieces of text with similar meaning end up as vectors that sit close together in that vector space. Similarity is usually measured with cosine similarity. Use cases worth naming: semantic search (finding results by meaning, not just keyword match), recommendation systems, clustering, and — most relevant here — retrieval in RAG systems.

RAG (Retrieval-Augmented Generation)

- Problem it solves: LLMs have a training cutoff and can hallucinate; RAG lets a model answer using current, private, or domain-specific data without retraining the model itself.
- Basic pipeline — be able to sketch this on paper if asked:
 - 1. Break source documents into smaller chunks.
 - 2. Convert each chunk into an embedding and store it in a vector database (e.g., Pinecone, FAISS, Chroma, Weaviate).
 - 3. When a user asks a question, embed the question too.
 - 4. Retrieve the most similar chunks from the vector store.
 - 5. Insert those retrieved chunks into the prompt as context.
 - 6. The LLM generates an answer grounded in that retrieved context, rather than from memory alone.

Likely question: *How would you build a system that answers employee questions using our internal company documents?*

- This is literally asking you to describe RAG — walk through the six-step pipeline above out loud.

Likely question: *What's the difference between RAG and fine-tuning?*

- Fine-tuning changes the model's internal weights using additional training data (more expensive, more effort, and it becomes stale again over time); RAG keeps the model as-is and feeds it fresh, relevant context at query time (cheaper, easier to keep updated, and you can trace which document an answer came from).

LLM Providers & AI Coding Tools (named directly in the JD)

- OpenAI — GPT models, the API, and ChatGPT.
- Anthropic — the Claude family of models and API.
- Google — Gemini models, and Vertex AI, Google Cloud's platform for building and deploying ML/AI models. (You've already used Vertex AI in FaithGuard — be ready to explain exactly what you used it for; if it was a smaller or exploratory part of the project, say so honestly rather than overstating it.)
- Azure AI — Microsoft's cloud AI services, which also hosts OpenAI's models for enterprise customers.
- Hugging Face — an open-source hub for models and datasets, plus the widely used transformers Python library for running open models yourself.
- AI coding tools — GitHub Copilot (inline code suggestions in your editor), Cursor (an AI-native code editor built around chat plus codebase context), Claude Code (an agentic tool that can read, plan, write, and run code changes across a codebase, from the terminal or an IDE). If you've used any of these — even ChatGPT or Copilot for coursework — mention it honestly; if not, say you understand what it does and are keen to pick it up quickly. That's a completely normal answer for a fresher-level role.

A few extra scenario questions worth rehearsing out loud

Likely question: *How would you stop an LLM from making things up?*

- Ground it with RAG, instruct it explicitly to say “I don't know” when uncertain, ask for citations/sources, and keep a human in the loop for high-stakes outputs.

Likely question: *What would you do if two retrieved chunks in a RAG system contradicted each other?*

- Surface both, prefer the more recent or authoritative source, or explicitly flag the conflict to the user rather than silently picking one.

Likely question: *Have you used an LLM API directly?*

- If not, it's fine to say so — but know conceptually that it's typically a simple HTTP POST request with your prompt in the body and the model's response coming back as JSON, similar in spirit to the REST APIs you've already used in your own projects (Google Maps API, Google Cloud Vision API). Drawing that parallel yourself is a strong answer.

6. Project Deep-Dive — Anticipated Questions on Your Own Projects

This is the single most valuable section to actually rehearse out loud. Interviewers almost always dig into listed projects, and these three are a large share of your resume's substance.

FaithGuard — AI-Assisted Lost & Found System

Likely question: *Walk me through the architecture — what talks to what?*

Likely question: *Why QR-based anonymous authentication instead of a normal login system? What problem does that solve, and what trade-offs come with it (e.g., account recovery, abuse prevention)?*

Likely question: *How does the real-time notification system actually work under the hood?*

- Be ready to explain, at a basic level, how Firebase Cloud Messaging gets a push notification to a user's device.

Likely question: *You listed Vertex AI in your tech stack — what specifically did you use it for?*

- Prepare a concrete, honest answer — even if it was a smaller or exploratory piece, explain what you tried and what you learned.

Likely question: *If this app suddenly had 100,000 users, what would break first, and how would you fix it?*

- A good answer just needs to show you understand scaling basics — database read/write limits, notification delivery at scale — it doesn't need to be a perfect answer, just a thoughtful one.

Cloud Image Recognition System

Likely question: *What's the difference between the OCR, object detection, and face/landmark detection features you built — technically, how does the Vision API treat each one differently?*

Likely question: *Why Tkinter for the GUI instead of a web front-end? What would you do differently if you rebuilt this today?*

Likely question: *How did you keep uploaded images secure in Cloud Storage, and how does Firebase Authentication tie into that?*

Likely question: *Which of the four features was hardest to get working reliably, and why?*

Sales Data Analysis Dashboard

Likely question: *Walk me through your exploratory data analysis process, start to finish.*

Likely question: *What was the most interesting or surprising insight you found in the sales data?*

Likely question: *Pick one of your SQL queries and explain it line by line.*

Likely question: *Why Power BI here specifically, given you also list Tableau as a skill? How did you decide which six visualizations made the final cut?*

Likely question: *How would this analysis change if the dataset were 100x larger?*

A grounding note for all three projects: the strongest possible answer format is — what the project does, then one specific technical decision you made and why, then one concrete challenge you hit and how you solved it, then what you'd improve with more time. That structure alone outperforms a generic feature-list recap, which is what most candidates default to.

7. Round 4 — Project Assignment

You don't know the exact format yet, so prepare your approach rather than trying to guess the exact task. Given the “AI Lab” focus and your own resume, realistic possibilities include: a small Python/Jupyter data task, a prompt-engineering exercise (write and refine prompts for a given use case), a short script involving an LLM API call, or a SQL/data-analysis task similar to your Sales Dashboard project.

How to approach any assignment you're given

- Clarify scope first. If anything is ambiguous, ask a clarifying question before writing code — this itself signals good engineering habits, not a weakness.
- Plan briefly before coding. Even 2–3 minutes sketching your approach, on paper or in comments, prevents backtracking later.
- Write clean, commented code. Assume someone else will read it — because someone will, when you explain it afterward.
- Test edge cases. Empty input, unexpected data, and so on — mention out loud that you're thinking about this even if you don't have time to handle every case.
- Manage your time. A working, simpler solution beats an incomplete, ambitious one. If you're running low on time, say so and explain what you'd do next given more time.
- Prepare to explain and defend it. Be ready to walk through your reasoning and trade-offs — this discussion often matters as much as the code itself.

8. How To Stand Out — Communication & Presence

How to “justify” your answers so they're convincing

- Never leave a claim unsupported. Every time you say something like “I'm a fast learner” or “I have good problem-solving skills,” immediately follow it with a specific example or number from your actual experience. Claim, then evidence, every time.
- Explain the “why,” not just the “what.” Anyone can say “I used Firebase.” A standout answer explains why you chose it over the alternative and what trade-off you accepted.
- Acknowledge trade-offs openly. Saying you chose X, though Y would have been better for a specific reason, but you didn't have time to implement Y, reads as mature engineering judgment, not weakness.
- Connect back to the role. Where it fits naturally, tie your answer back to what an AI Lab Intern would actually need to do — this shows you've thought about the job, not just memorized answers about yourself.

General standout habits

- Lead with the direct answer, then explain — don't make the interviewer wait for your point.
- Use real numbers wherever you have them — you already have plenty: 5+ modules, 4+ features, 10+ queries, 6+ visualizations, 30+ members.
- If you don't know something, say so plainly, then show how you'd figure it out — for example, you haven't used FAISS specifically, but understand it's a vector similarity search library, and would start with its documentation and a small test dataset. This is far stronger than guessing or freezing.
- Ask one thoughtful clarifying question during the technical rounds if something is genuinely ambiguous — it signals real engineering instinct.
- Show genuine curiosity about AI/LLMs specifically, since that's what this role is actually about — not just “I want a job,” but “I want to build in this specific space.”

9. Smart Questions To Ask Them

Always have two or three ready — it signals genuine interest, not just eagerness to be hired.

- What tools and frameworks does the AI Lab team currently work with day to day?
- What would a typical first month look like for someone in this role?
- What mentorship or learning support is available during the internship?
- Could you walk me through what the 3-year service agreement covers in practice — for example, is there an exit or buyout clause if my circumstances change? (This is a completely standard, professional question. Asking it doesn't hurt your candidacy, and you deserve full clarity on a 3-year commitment before you sign anything.)
- What does the path from intern to full-time look like, and how is that transition evaluated?

10. Final Night-Before / Morning-Of Cheat Sheet

Your 30-second pitch

3rd-year CSE @ ADCET (CPI 8.81) → full-stack + data analytics + growing AI/ML (Google AI-ML internship) → 3 shipped projects → founded a 30-member community.

RAG in one breath

Chunk documents → embed → store in a vector DB → retrieve relevant chunks for a query → feed them to the LLM as context → grounded answer.

Prompt engineering in one breath

Zero-shot vs few-shot, chain-of-thought, be specific, specify the output format, iterate.

Your three projects, one line each

- FaithGuard — PWA, QR-based anonymous auth, Firebase + Vertex AI + Google Maps API.
- Cloud Image Recognition System — Tkinter GUI, Google Cloud Vision API, 4+ image-analysis features.
- Sales Data Analysis Dashboard — Python/Pandas/SQL feeding an interactive Power BI dashboard, 6+ visualizations.

Two rules to remember walking in

- Every claim needs a number or example attached to it.
- Bring: resume copies, college ID, academic documents, formal attire, pen + notepad.

Breathe. You already have three real, working projects and a live AI/ML internship — most fresher candidates walking in tomorrow won't have either. Good luck.